

Commentary Engine

Technical Deep Dive

FastAPI · Bootstrap 5 · Azure OpenAI · LangGraph · SQLite

Document	Commentary Engine — Full Technical Reference
Version	1.0
Prepared by	Genpact · Nissan Partnership
Date	14 May 2026
Classification	Confidential & Proprietary
Stack	Python 3.11 · FastAPI · Azure OpenAI · LangGraph · SQLite · Bootstrap 5

#	Section	Topic
1	Project Overview	What the tool does, who uses it, key capabilities
2	Streamlit → FastAPI Migration	Why we migrated and how the architecture changed
3	Project Structure	Directory layout, file responsibilities, module boundaries
4	FastAPI Application Layer	main.py, routers, endpoints, session management
5	Services Layer	data_helpers, excel_export, session_store, ai_engine
6	Frontend Architecture	SPA design, tab modules, core.js, CSS system
7	Data Pipeline	Upload → normalise → filter → pivot → display flow
8	AI & LLM Integration	Azure OpenAI, LangGraph agent, persona system
9	Database Design	SQLite schema, XLSX storage, session store, SQL queries
10	API Reference	All 28 endpoints across 7 routers with request/response

11	Bug Fixes & Key Decisions	NaN JSON fix, itertuples fix, session key fix
12	Deployment Guide	Azure App Service, Docker, CI/CD, environment vars

1. Project Overview

What the Commentary Engine does and why it exists

The **Commentary Engine** is a full-stack financial analytics web application built for the Genpact–Nissan Finance Partnership. It replaces a manual, Excel-based variance commentary process with an automated, AI-assisted platform. Finance teams upload monthly data files, run scenario variance analysis, generate board-ready commentary using Azure OpenAI, and maintain a searchable historical knowledge base of past commentary — all through a single browser interface.

1.1 Core Capabilities

Tab 1 — Tagetik Mapping	Upload monthly Excel → merge entity/cost-category mapping → export enriched dataset
Tab 2 — Variance Analysis	Filter by scenario, region, division → pivot table with Variance (M€) and Variance % → Excel export
Tab 3 — Commentary Generator	Select audience persona → run LangGraph AI agent → get Exec Summary + RCA + Category Commentary → export .md/.txt/.pptx
Tab 4 — Chat with Data	Natural-language Q&A; over loaded dataset → persona-aware suggested prompts → answer stored in SQLite
Tab 5 — Run History	Browse all past AI analysis runs → view full summaries → rate with 🟢/🔴 feedback
Tab 6 — Comment Search	Filter master commentary database by 8 dimensions → free-text search → download filtered results
Tab 7 — PPT Upload	Upload SVP PowerPoint deck → extract slide text → enrich with metadata → push to master database

1.2 Tech Stack Summary

Layer	Technology	Purpose
Frontend	HTML · CSS · Bootstrap 5 · Vanilla JS	SPA shell, 7 tab modules, responsive UI
Backend	Python 3.11 · FastAPI · Uvicorn	REST API, routing, request handling
Data Processing	Pandas 2.2 · NumPy 1.26 · openpyxl 3.1	Excel I/O, pivot, variance computation
AI / LLM	Azure OpenAI GPT-4 · LangChain · LangGraph	Commentary generation, chat agent
PPT Processing	python-pptx 0.6	PPT extraction and deck generation
Session State	In-memory dict (session_store.py)	Per-user state via va_sid cookie
Database	SQLite · Segregateddata.xlsx · mapping.xlsx	Run history, chat logs, master commentary
Templates	Jinja2 3.1	Server-side HTML rendering (SPA shell)
Deployment	Azure App Service · Docker · GitHub Actions	CI/CD pipeline, production hosting

2. Streamlit → FastAPI Migration

Why we migrated and how the architecture fundamentally changed

2.1 Why We Migrated

The original application was built in **Streamlit** — a Python-only framework that renders UI server-side and re-runs the entire script on every user interaction. While ideal for rapid prototyping, Streamlit has fundamental limitations for a production finance tool used by multiple concurrent users.

Streamlit Problem	FastAPI Solution
Full script re-runs on every interaction — poor performance with large dataframes	API endpoints only execute the requested logic. Frontend holds state in session storage
No real URL routing — single-page with widget state only	7 clean REST routers (/api/tab1 ... /api/tab7) with proper HTTP semantics
Session state shared across browser tabs (st.session_state is process-global)	Per-session keyed by va_sid cookie — fully isolated sessions
Cannot serve a custom branded HTML/CSS frontend	Jinja2 templates + full Bootstrap 5 CSS control over every pixel
st.cache_data works only in single-process mode	services/ layer is pure Python — cacheable, testable, reusable
Difficult to test — tightly coupled to Streamlit runtime	FastAPI TestClient + pytest — 100% testable without a browser
No WebSocket or streaming support for chat	async endpoints + run_in_executor for non-blocking LLM calls

2.2 Architectural Shift

The migration was not a line-by-line rewrite. It required rethinking the separation of concerns entirely. In Streamlit, UI logic, business logic, and state management are all interleaved in a single Python file per tab. In FastAPI, these are cleanly separated into three layers:

- **Layer 1 — Frontend (static/js):** Each tab is an IIFE (Immediately Invoked Function Expression) JavaScript module. It owns its HTML template, event handlers, API calls, and DOM rendering. It talks to the backend via fetch().
- **Layer 2 — Backend (routers/):** Each FastAPI router maps HTTP verbs to Python functions. It validates input, calls the services layer, stores results in session, and returns JSON.
- **Layer 3 — Services (services/):** Pure Python — no FastAPI, no Streamlit, no browser dependency. data_helpers.py, excel_export.py, session_store.py, ai_engine.py. Fully unit-testable.

2.3 Session State Replacement

Streamlit's st.session_state is replaced by services/session_store.py — a thread-safe in-memory dictionary keyed by a UUID stored in the browser cookie va_sid. Every API request reads the cookie, looks up the session, and can read or write data. For production, this is designed to be swapped to Redis with minimal code change.

```
# session_store.py - core pattern
_store: dict[str, dict[str, Any]] = {} # {session_id: {key: value}}
def get(sid, key, default=None):
    with _lock:
        return _store.get(sid, {}).get(key, default)
def set(sid, key, value):
    with _lock:
        _store.setdefault(sid, {})[key] = value
# Production upgrade: replace with Redis #
```

```
redis_client.set(f'{sid}:{key}', json.dumps(value), ex=3600)
```

3. Project Structure

Directory layout, file responsibilities and module boundaries

```
fastapi_app/ ■■■■ main.py # FastAPI app, router registration, SPA fallback ■■■■ requirements.txt # All
Python dependencies ■■■■ .env.example # Environment variable template ■■■■ routers/ # API layer – one
file per tab ■■■■ tab1_mapping.py # POST /upload GET /download/csv|xlsx ■■■■ tab2_variance.py # POST
/upload|run|load-tab1 GET /download ■■■■ tab3_commentary.py # POST /run {persona} GET
/download/md|txt|pptx ■■■■ tab4_chat.py # POST /ask DELETE /clear ■■■■ tab5_history.py # GET /runs
POST /runs/{id}/feedback ■■■■ tab6_search.py # GET /filters POST /search GET /download ■■■■
tab7_ppt.py # POST /upload|push GET /download|master ■■■■ services/ # Pure Python business logic ■■■■
data_helpers.py # Excel I/O, normalise, filter, pivot, variance ■■■■ excel_export.py # Multi-sheet .xlsx
builder (openpyxl) ■■■■ session_store.py # In-memory session (→ Redis in prod) ■■■■ ai_engine.py #
LangGraph agent, PPTX generator, Azure check ■■■■ templates/ ■■■■ index.html # Jinja2 SPA shell –
Bootstrap 5, 7 tab panels ■■■■ static/ ■■■■ css/app.css # Full brand CSS – dark header, pivot, upload
zones ■■■■ js/ ■■■■ core.js # Tab switch, toast, VaMultiselect, vaPost/vaGet ■■■■ tab1.js ... tab7.js
# Tab-specific UI logic ■■■■ backend/ ■■■■ mapping.xlsx # Entity→Region/Market, CostCat→OH/LC lookup
■■■■ database/ ■■■■ Segregateddata.xlsx # Master commentary store (Tab 6 search, Tab 7 push)
```

■ The routers/ and services/ directories are intentionally decoupled. A router never imports from another router. A service never imports from a router. This ensures each layer is independently testable and replaceable.

4. FastAPI Application Layer

main.py, routers, endpoint design, session management

4.1 main.py — The Entry Point

main.py is intentionally thin. It creates the FastAPI app, mounts static files, configures Jinja2 templates, and registers all 7 routers. Business logic lives exclusively in the services layer.

```
from fastapi import FastAPI from fastapi.staticfiles import StaticFiles from fastapi.templating import Jinja2Templates app = FastAPI(title='Commentary Engine', version='1.0.0') app.mount('/static', StaticFiles(directory='static'), name='static') templates = Jinja2Templates(directory='templates') # Register all 7 routers app.include_router(tab1_mapping.router, prefix='/api/tab1') app.include_router(tab2_variance.router, prefix='/api/tab2') app.include_router(tab3_commentary.router, prefix='/api/tab3') app.include_router(tab4_chat.router, prefix='/api/tab4') app.include_router(tab5_history.router, prefix='/api/tab5') app.include_router(tab6_search.router, prefix='/api/tab6') app.include_router(tab7_ppt.router, prefix='/api/tab7') # SPA fallback – every non-API GET returns index.html @app.get("/{full_path:path}") async def spa_fallback(request, full_path=''): return templates.TemplateResponse('index.html', {'request': request})
```

4.2 Router Design Pattern

Every router follows the same pattern: extract the session ID from the cookie, load or store data from session_store, call the appropriate service, sanitise the response for JSON safety, and return JSONResponse. This consistency makes every router predictable and easy to extend.

```
# Standard router pattern (tab2_variance.py example) @router.post('/run') async def run_variance(request: Request): sid = request.cookies.get('va_sid', str(uuid.uuid4())) # session identity df = _load_master(sid) # read from session store body = await request.json() # parse request body # Call pure Python service layer leaf_df, pivot_source, period = dh.run_variance( df_filtered, group_fields, scenario_a, scenario_b, sel_period, month_cols ) leaf_df = _sanitise_df(leaf_df) # NaN/inf → None before JSON ss.set(sid, 'leaf_df_bytes', leaf_df.to_json(orient='split')) # store result return JSONResponse({ # return safe JSON 'records': _safe_records(leaf_df), 'total_variance': fmt_num(total_var), ... })
```

4.3 Session Management

Each user gets a UUID session ID stored in a browser cookie named va_sid. The session store maps this ID to a dictionary of named data blobs (DataFrames serialised as JSON orient='split', analysis results, etc.). The session is automatically created on the first API call and persists until the server restarts or is explicitly cleared.

Session Key	Set By	Read By	Contents
final_db_bytes	Tab 1 upload	Tab 2 load-tab1, Tab 4	Mapped DataFrame (JSON split)
master_db_bytes	Tab 2 upload	Tab 2 run, download	Master DataFrame for variance
leaf_df_bytes	Tab 2 run	Tab 2 download, Tab 3	Pivot result DataFrame
pivot_src_bytes	Tab 2 run	Tab 2 download	Long-form aggregation
var_context	Tab 2 run	Tab 2 download	Scenario names, group fields
cg_summary	Tab 3 run	Tab 3 downloads	Full AI commentary text

cg_persona	Tab 3 run	Tab 3 PPTX download	Persona ID used
chat_history	Tab 4 ask	Tab 4 display	List of {role, content}
search_result_bytes	Tab 6 search	Tab 6 download	Filtered commentary DF
ppt_extract_bytes	Tab 7 upload	Tab 7 push, download	Extracted PPT DataFrame

5. Services Layer

data_helpers · excel_export · session_store · ai_engine

5.1 data_helpers.py — The Core Data Pipeline

This is the most critical service file. It handles everything from reading raw Excel bytes to producing the final pivot DataFrame that the frontend renders. It has zero FastAPI or Streamlit imports — it is pure Python, fully unit-testable.

Function	Input	Output	Description
read_and_normalise(bytes)	Raw .xlsx bytes	pd.DataFrame	Read Excel, normalise month headers (1-Apr, 2-May...)
generate_mapping(bytes)	Raw .xlsx bytes	pd.DataFrame	Full Tab 1 pipeline: read → MTD/YTD → merge mapping.x
get_filter_options(df)	pd.DataFrame	dict	Extract unique values for all 6 filter dropdowns
apply_filters(df, ...)	DataFrame + filter params	pd.DataFrame	Apply all sidebar filters, restrict to two scenarios
run_variance(df, ...)	Filtered DF + params	(leaf_df, src, label)	Aggregate → pivot_table → compute delta and delta_pct
_format_leaf_df(df, scA, scB)	pivot DF + names	pd.DataFrame	Rename A/B → scenario names, convert to M€, % format
_safe_records(df)	pd.DataFrame	list[dict]	iloc-based serialisation — safe for columns with spaces
_sanitise_df(df)	pd.DataFrame	pd.DataFrame	Replace NaN/inf in all columns before JSON serialisation
fmt_num(v)	float	str	Format number: 1234567 → '1.23M', 1234 → '1.2K'

5.2 The run_variance() Pipeline

This function is the heart of Tab 2. It takes a filtered DataFrame and produces the pivot result that the frontend renders as an expandable table.

```
def run_variance(df, group_fields, scenario_a, scenario_b, sel_period, month_cols): # Step 1: Compute the
value column if sel_period == '__YTD_CALC__': df['__VALUE__'] =
df[month_cols].apply(clean_numeric_series).sum(axis=1) value_col = '__VALUE__' else: value_col = sel_period
# e.g. '3-Jun' # Step 2: Aggregate by group + scenario agg = df.groupby(group_fields +
['Scenario'])['Value'].sum().reset_index() # Step 3: Pivot - rows=group_fields, columns=scenarios,
values=sum piv = agg.pivot_table(index=group_fields, columns='Scenario', values='Value', aggfunc='sum',
fill_value=0.0) # Step 4: Rename internal columns, compute delta piv = piv.rename(columns={scenario_a: 'A',
scenario_b: 'B'}) leaf_df = piv.reset_index() leaf_df['delta'] = leaf_df['A'] - leaf_df['B']
leaf_df['delta_pct'] = (leaf_df['delta'] / leaf_df['B']) # ratio return leaf_df, pivot_source_long,
period_label
```

5.3 ai_engine.py — LangGraph Agent

The AI engine uses LangGraph — a stateful graph execution framework built on top of LangChain. The agent has two nodes connected in sequence. The first node runs pure Python drill-down logic to build a variance tree. The second node calls Azure OpenAI with the tree data and a persona-specific system prompt.

```
class AgentState(TypedDict): df: pd.DataFrame hierarchy_cols: List[str] persona: str # NEW: CFO /
VP_FINANCE / AUDITOR etc. path_trace: List[str] final_level_data: List[str] tree_data: List[Dict[str, Any]]
```

```
final_summary: str # Graph: START → calculate_variance_node → synthesize_insight_node → END
def build_graph(): workflow = StateGraph(AgentState) workflow.add_node('calculate_variance', calculate_variance_node) workflow.add_node('synthesize_insight', synthesize_insight_node) workflow.add_edge(START, 'calculate_variance') workflow.add_edge('calculate_variance', 'synthesize_insight') workflow.add_edge('synthesize_insight', END) return workflow.compile() # Persona-aware system prompt in synthesize_insight_node: persona_context = PERSONA_PROMPTS.get(persona, DEFAULT_PERSONA_PROMPT) system_prompt = f'{persona_context}\n\n[structured output format]' response = llm.invoke([SystemMessage(content=system_prompt), HumanMessage(content='\n'.join(final_level_data))])
```

6. Frontend Architecture

SPA design, tab modules, core.js, CSS system

6.1 Single-Page Application Design

The frontend is a **vanilla JavaScript SPA** — no React, no Vue, no build step. index.html is served by FastAPI/Jinja2 as the shell. All 7 tab panels are pre-defined as empty <div id='tabN'> elements. Each tab JS file is an IIFE that renders its HTML into its div on first activation and is triggered by the va:tabchange custom event.

```

// core.js – tab switching
tabs.forEach(t => t.addEventListener('click', () => {
  panels.forEach(p => p.classList.toggle('active', p.id === t.dataset.tab));
  window.dispatchEvent(new CustomEvent('va:tabchange', { detail: t.dataset.tab }));
}));
// tab1.js – lazy init pattern
(function() {
  let initialised = false;
  function init() {
    if (initialised) return;
    initialised = true;
    document.getElementById('tab1-content').innerHTML = buildHTML();
    bindEvents();
  }
  window.addEventListener('va:tabchange', e => {
    if (e.detail === 'tab1') init();
  });
  window.addEventListener('DOMContentLoaded', init);
  // init default active tab
})();

```

6.2 core.js Shared Utilities

Utility	Description
VaMultiselect class	Custom multi-select dropdown with checkboxes. Used for all filter fields.
vaPost(url, body)	Wrapper around fetch() POST with JSON body. Throws on non-200 with error message.
vaGet(url)	Wrapper around fetch() GET. Used for filter loading and history fetching.
vaPostForm(url, fd)	Wrapper for multipart/form-data file uploads.
vaBuildTable(cols, records, opts)	Build a element from columns+records. Supports right-align, fav/adv colouring.
vaToast(msg, type)	Bootstrap 5 toast notification. Called after every successful API operation.
vaAlert(id, msg, type)	Show/hide inline alert div with error, warning, info, success styling.
vaProgress(id, pct, label)	Animate a progress bar. Used during file upload and analysis runs.
vaSpinner(label)	Return HTML string with animated CSS spinner for loading states.

6.3 Persona Bridge (Tab 3 → Tab 4)

The persona selected in Tab 3 (Commentary Generator) automatically changes the suggested prompts in Tab 4 (Chat with Data). This is implemented via a browser global variable window.__va_persona — the simplest possible inter-tab communication without a state management library.

```

// tab3.js – writes persona on every card click
function refreshPersonaUI() {
  window.__va_persona = selectedPersona.id;
  // 'CF0' | 'VP_FINANCE' | ... // update card UI...
}
// tab4.js – reads persona when tab activates
function getCurrentPersona() {
  const pid = window.__va_persona || 'CF0';
  return PERSONA_PROMPTS[pid] || PERSONA_PROMPTS.CF0;
}
// 6 personas × 4 prompt groups × 4 prompts each = 96 curated prompts
// CF0 → P&L; Impact, Risk, Strategic Actions, Executive Summary
// Auditor → Materiality, Control Checks, Evidence Trail, JSOX/Compliance

```

7. Data Pipeline

Upload → normalise → filter → pivot → Variance(M€) display

7.1 Month Header Normalisation

Excel exports from Tagetik use inconsistent date column formats: some files use '01-Apr', others use 'April', others use datetime objects. The `normalize_month_header()` function unifies all formats to the canonical form N-Mon (e.g. '1-Apr', '12-Mar') using regex.

```
MONTH_HEADER_REGEX = r'\d{1,2}-(Jan|Feb|Mar|Apr|May|Jun|Jul|Aug|Sep|Oct|Nov|Dec)'
def normalize_month_header(col) -> str:
    if hasattr(col, 'strftime'): # datetime object return
        f'{col.day}-{col.strftime("%b")}'
    s = str(col).strip() # matches '01-Apr', '1 Apr', '1/Apr', 'April', 'apr'
    etc. m = re.match(r'^0?(\d+)\s*[-/] \s*([A-Za-z]+)$', s)
    if m: return f'{int(m.group(1))}-{FULL_TO_ABBR.get(m.group(2).capitalize(), m.group(2)[:3].title())}'
    return s
```

7.2 Variance Computation & Display Formatting

Raw numbers from Tagetik are in absolute values (euros). The display formatting pipeline converts them to millions with 2 decimal places and converts the delta ratio to a percentage — matching how finance teams think and present numbers.

```
def _format_leaf_df(df, scenario_a, scenario_b):
    # Convert raw euros → M€
    df['A'] = (df['A'] / 1_000_000).round(2)
    df['B'] = (df['B'] / 1_000_000).round(2)
    df['delta'] = (df['delta'] / 1_000_000).round(2)
    # delta_pct is a ratio (0.02 = 2%) → convert to % float (2.00)
    df['delta_pct'] = (df['delta'] * 100).round(2)
    # Rename to scenario names + display labels
    return df.rename(columns={
        'A': scenario_a, # e.g. 'Budget 2025'
        'B': scenario_b, # e.g. 'Actual YTD'
        'delta': 'Variance (M€)',
        'delta_pct': 'Variance %',
    })
```

7.3 Expandable Pivot Table (Frontend)

The pivot table in Tab 2 is built entirely in JavaScript using DOM manipulation — no library required. Rows are grouped by the first hierarchy field. Clicking a group row toggles its detail rows, creating an Excel-like expand/collapse behaviour.

```
// Build group totals in JS (client-side aggregation)
records.forEach(r => {
    const key = r[firstDim] ?? '(blank)';
    if (!groupTotals[key]) groupTotals[key] = { scA: 0, scB: 0, rows: [] };
    groupTotals[key].scA += (r[scA] || 0);
    groupTotals[key].scB += (r[scB] || 0);
    groupTotals[key].rows.push(r);
});

// Group header row click → toggle detail rows
grpRow.addEventListener('click', () => {
    const isOpen = grpRow.classList.toggle('open');
    tbody.querySelectorAll(`tr[data-parent='${groupId}']`)
        .forEach(dr => dr.style.display = isOpen ? '' : 'none');
});
```

8. AI & LLM Integration

Azure OpenAI · LangGraph · Persona System · Pandas Agent

8.1 LangGraph Agent Architecture

LangGraph implements the commentary generation as a **stateful directed graph**. This is superior to a simple function call because: state is explicitly typed via TypedDict, nodes are independently testable, and the graph can be extended with conditional edges, parallel branches, or human-in-the-loop steps.

LangGraph execution flow

- 1. `calculate_variance_node(state)`: Groups data by hierarchy, computes top-5 per level, builds a tree of {column, item, value, children}. Pure Python — no LLM call.
- 2. `synthesize_insight_node(state)`: Takes `final_level_data` list from node 1, constructs persona-aware `SystemMessage` + `HumanMessage`, calls `AzureChatOpenAI`, returns `final_summary` string.
- 3. The compiled graph is invoked with `app_graph.invoke(inputs)` — LangGraph handles state passing between nodes automatically.

8.2 Persona System

Six distinct audience personas are defined, each with a different system prompt prefix that instructs GPT-4 to adopt a specific communication style, vocabulary level, and emphasis framework.

Persona ID	Role	Tone & Focus
CFO	Chief Financial Officer	Strategic · P&L impact · risk-aware · board-ready · decisive language
VP_FINANCE	VP Finance	Budget stewardship · cost-control · FP&A framing · corrective actions
BUSINESS_PARTNER	Finance Business Partner	Operational root-cause · action-oriented · team-specific · practical
BOARD	Board / Executive	High-level narrative · investor-ready · max 3 key messages · governance
AUDITOR	Internal Auditor	JSOX-compliant · evidence-based · materiality thresholds · control review
FP_A	FP&A Analyst	Data-driven · driver trees · model-backed · technical terminology

8.3 Pandas DataFrame Agent (Tab 4 Chat)

The chat tab uses LangChain's `create_pandas_dataframe_agent()` which wraps GPT-4 with Python code execution tools. The agent can write and execute pandas code against the loaded DataFrame to answer any data question. Because this is a blocking operation, it runs in a thread pool to avoid blocking FastAPI's async event loop.

```
async def _run_agent(df: pd.DataFrame, question: str) -> str: llm = AzureChatOpenAI(temperature=0, ...) #
deterministic responses agent = create_pandas_dataframe_agent( llm, df, agent_type='openai-functions', #
tool-calling mode allow_dangerous_code=True, # required for pd exec verbose=False, ) # Run in thread pool -
never block the async event loop loop = asyncio.get_event_loop() result = await loop.run_in_executor( None,
lambda: agent.invoke({'input': question}) ) return result.get('output', str(result))
```

9. Database Design

SQLite schema · XLSX master store · SQL queries · feedback loop

9.1 SQLite — analysis_history.db

SQLite is used for structured persistence of AI analysis runs and chat logs. It is a file-based database requiring no server — ideal for single-server deployment. For multi-server Azure deployment, this would migrate to Azure SQL.

```
-- Table 1: AI commentary runs (Tab 3) CREATE TABLE IF NOT EXISTS runs ( id INTEGER PRIMARY KEY
AUTOINCREMENT, timestamp TEXT, -- '2025-01-15 14:32:11' filename TEXT, -- source file name hierarchy TEXT,
-- JSON array: '["OH/LC","Division_Desc"]' total_variance TEXT, -- '2.34M' summary TEXT, -- full
AI-generated commentary feedback INTEGER DEFAULT 0 -- 1=thumbs up, -1=thumbs down, 0=none ); -- Table 2:
Chat Q&A; history (Tab 4) CREATE TABLE IF NOT EXISTS chat_logs ( id INTEGER PRIMARY KEY AUTOINCREMENT,
timestamp TEXT, prompt TEXT, -- user's question response TEXT, -- AI answer feedback INTEGER DEFAULT 0 );
```

9.2 SQL Queries Used

Operation	SQL	Used In
Insert run	INSERT INTO runs (timestamp,filename,hierarchy,total_variance,summary,feedback) VALUES (?,?,?,?,?,?)	Tab 3
List all runs	SELECT id,timestamp,filename,total_variance,feedback FROM runs ORDER BY timestamp DESC	Tab 5
Get one run	SELECT * FROM runs WHERE id=?	Tab 5 click row
Update feedback	UPDATE runs SET feedback=? WHERE id=?	Tab 3  , Tab 5
Insert chat	INSERT INTO chat_logs (timestamp,prompt,response) VALUES (?,?,?)	Tab 4, after each answer
Feedback summary	SELECT feedback FROM runs	Tab 5 summary counts

9.3 XLSX Storage — Segregateddata.xlsx

The master commentary database is an Excel file rather than a relational table. This was a deliberate choice to allow finance users to open, edit, and share the database without any database tooling. Tab 6 filters in-memory using pandas after each request. Tab 7 appends to it and deduplicates by Slide_Number + Comments.

```
# Tab 7 push logic master = pd.read_excel(MASTER_XLSX) # read existing combined = pd.concat([master,
df_new], ignore_index=True) # append combined = combined.drop_duplicates( # deduplicate
subset=['Slide_Number', 'Comments'], keep='last' ).reset_index(drop=True) combined.to_excel(MASTER_XLSX,
index=False) # overwrite # Tab 6 search logic df = pd.read_excel(MASTER_XLSX) # read fresh every request df
= _sanitise_df(df) # clean NaN/inf for col, vals in filters.items(): # apply column filters df =
df[df[col].isin(vals)] df = df[df['Comments'].str.contains(q, case=False, na=False)] # keyword
```

9.4 Feedback Loop Design

Every AI output has a  rating mechanism. Feedback is stored in SQLite and visible in Tab 5. This creates a data flywheel: over time, low-rated commentary can be used to fine-tune persona prompts or identify hierarchy

configurations that produce poor results.

Feedback loop flow

- User generates commentary in Tab 3 → run saved to SQLite with feedback=0
- User clicks 🍷 or 🍷 → POST /api/tab3/runs/{id}/feedback {score: 1 or -1}
- Router calls: UPDATE runs SET feedback=? WHERE id=?
- Tab 5 Run History shows all runs with feedback emoji (🍷/🍷/🍷)
- Tab 5 summary: SELECT feedback FROM runs → count positive/negative/none
- Same pattern for Tab 4 chat answers stored in chat_logs table

10. API Reference

All 28 endpoints across 7 routers

Method	Endpoint	Body / Params	Returns
POST	/api/tab1/upload	file: UploadFile (.xlsx)	rows, columns, preview[100], warning
GET	/api/tab1/download/csv	—	text/csv stream
GET	/api/tab1/download/xlsx	—	application/xlsx stream
POST	/api/tab2/upload	file: UploadFile	rows, scenarios, filter options
POST	/api/tab2/upload-two	file_a, file_b, label_a, label_b	Combined DF options
GET	/api/tab2/filters	—	scenarios, markets, regions, avail_c
POST	/api/tab2/load-tab1	—	Promotes Tab1 output → Tab2 session
POST	/api/tab2/run	scenario_a/b, sel_period, group_filters, filters, totals, top5, bot5	files, df, totals
GET	/api/tab2/download/xlsx	—	Variance report .xlsx stream
POST	/api/tab3/run	{persona, hierarchy_cols, variance_path}	trace, tree_data, final_summary
GET	/api/tab3/download/md	—	text/markdown stream
GET	/api/tab3/download/txt	—	text/plain stream
GET	/api/tab3/download/pptx	—	PowerPoint .pptx stream
POST	/api/tab4/ask	{question: string}	answer, history[]
DELETE	/api/tab4/clear	—	{status: cleared}
GET	/api/tab5/runs	—	List of runs with feedback_label
GET	/api/tab5/runs/{id}	—	Full run with hierarchy_list
POST	/api/tab5/runs/{id}/feedback	{score: 1 -1}	{status, run_id, score}
GET	/api/tab5/summary	—	{positive, negative, none}
GET	/api/tab6/filters	—	filter_cols, options{}, total_rows
POST	/api/tab6/search	{filters{}, search_text}	columns, records[500], count
GET	/api/tab6/download/csv	—	text/csv stream
GET	/api/tab6/download/xlsx	—	application/xlsx stream
POST	/api/tab7/upload	file: UploadFile (.pptx/.ppt)	rows, columns, records
POST	/api/tab7/push	—	{status, pushed_rows, master_rows}
GET	/api/tab7/download	—	Edited .xlsx stream
GET	/api/tab7/master	—	columns, records[100], total

11. Bug Fixes & Key Technical Decisions

NaN JSON · itertuples crash · session key mismatch · footer fix

11.1 NaN / inf JSON Serialisation (Critical Fix)

The most common runtime error was `ValueError: Out of range float values are not JSON compliant: nan`. This occurs because Python's `json` module (used by FastAPI's `JSONResponse`) rejects NaN and inf — which appear in pandas DataFrames from empty cells, divide-by-zero, and numpy type mismatches.

```
# WRONG — df.where().to_dict() misses numpy.float64('nan') records = df.where(pd.notnull(df),
None).to_dict(orient='records') # CORRECT — explicit check per value using math module
def _safe_val(v):
    if v is None: return None
    if isinstance(v, float) and (math.isnan(v) or math.isinf(v)): return None
    try:
        import numpy as np
        if isinstance(v, np.floating): f = float(v)
        return None if (math.isnan(f) or math.isinf(f)) else f
    except ImportError: pass
    return v
def _safe_records(df):
    # iloc — never itertuples
    cols = df.columns.tolist()
    return [{col: _safe_val(df.iloc[i, j]) for j, col in enumerate(cols)} for i in range(len(df))]
```

11.2 itertuples Crash on Column Names with Spaces

`pandas DataFrame.itertuples()` converts column names to valid Python identifiers by replacing spaces with underscores and numbers. A column named 'Slide Number' becomes `_1` in the named tuple — so `getattr(row, 'Slide Number')` raises `AttributeError`.

```
# WRONG — 'Slide Number' → '_1' in namedtuple
for row in df.itertuples(index=False):
    val = getattr(row, 'Slide Number')
# AttributeError!
# CORRECT — iloc uses string column names directly
for i in range(len(df)):
    for j, col in enumerate(df.columns):
        val = df.iloc[i, j]
# always works
# Also renamed 'Slide Number' → 'Slide_Number' in _extract_ppt()
# Belt-and-braces: both old and new column names handled in deduplication
```

11.3 Session Key Mismatch (Tab 1 → Tab 2)

Tab 1 stored the mapping output under the key 'final_db_bytes'. Tab 2 only looked for 'master_db_bytes'. Result: 'Load Tab 1 Data' always returned 404. Fixed by making `_load_master()` fall back to the Tab 1 key, and adding a dedicated POST `/api/tab2/load-tab1` endpoint that explicitly promotes the key.

```
def _load_master(sid):
    raw = ss.get(sid, 'master_db_bytes')
    # try Tab 2 first
    if raw is None:
        raw = ss.get(sid, 'final_db_bytes')
    # fall back to Tab 1
    if raw is None:
        raise HTTPException(404, 'No data in session...')
    return pd.read_json(io.StringIO(raw), orient='split')
@router.post('/load-tab1')
async def load_tab1(request):
    raw = ss.get(sid, 'final_db_bytes')
    df = pd.read_json(io.StringIO(raw), orient='split')
    ss.set(sid, 'master_db_bytes', df.to_json(orient='split'))
    # promote
    return JSONResponse(**dh.get_filter_options(df), 'source': 'tab1')
```

12. Deployment Guide

Local development · Docker · Azure App Service · CI/CD

12.1 Local Development

```
# 1. Create virtual environment python -m venv venv source venv/bin/activate # Windows:
venv\Scripts\activate # 2. Install dependencies pip install -r requirements.txt # 3. Configure environment
cp .env.example .env # Edit .env: add AZURE_OPENAI_ENDPOINT, KEY, DEPLOYMENT, API_VERSION # 4. Create
required folders mkdir -p backend database # Place mapping.xlsx in backend/ (optional) # 5. Run the server
uvicorn main:app --reload --port 8000 # 6. Open browser # http://localhost:8000 # API docs:
http://localhost:8000/docs
```

12.2 Docker

```
# Dockerfile FROM python:3.11-slim WORKDIR /app COPY requirements.txt . RUN pip install --no-cache-dir -r
requirements.txt COPY . . EXPOSE 8000 CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "
--workers", "2"] # Build and run docker build -t commentary-engine:latest . docker run -p 8000:8000
--env-file .env commentary-engine
```

12.3 Azure App Service Deployment

```
# 1. Create Azure resources az group create --name rg-commentary --location eastus az acr create --name
commentaryacr --sku Basic --resource-group rg-commentary az appservice plan create -n commentary-plan -g
rg-commentary --sku B2 --is-linux az webapp create -n commentary-engine -g rg-commentary \ -p
commentary-plan --runtime 'PYTHON:3.11' # 2. Push Docker image to ACR az acr login --name commentaryacr
docker tag commentary-engine:latest commentaryacr.azurecr.io/commentary-engine:v1.0 docker push
commentaryacr.azurecr.io/commentary-engine:v1.0 # 3. Configure App Service az webapp config appsettings set
-n commentary-engine -g rg-commentary --settings \ AZURE_OPENAI_ENDPOINT=https://... \
AZURE_OPENAI_DEPLOYMENT=gpt-4 \ AZURE_OPENAI_API_VERSION=2024-02-15-preview \
AZURE_OPENAI_KEY=@Microsoft.KeyVault(SecretUri=https://...) \ WEBSITES_PORT=8000 # 4. Set startup command
az webapp config set --startup-file \ 'uvicorn main:app --host 0.0.0.0 --port 8000 --workers 2'
```

12.4 Environment Variables

Variable	Required	Description
AZURE_OPENAI_ENDPOINT	Yes	https://your-resource.openai.azure.com/
AZURE_OPENAI_KEY	Yes	API key — use Key Vault reference in production
AZURE_OPENAI_DEPLOYMENT	Yes	Deployment name (e.g. 'gpt-4')
AZURE_OPENAI_API_VERSION	Yes	e.g. '2024-02-15-preview'
REDIS_URL	No	Redis connection string (replaces in-memory session store for multi-instance prod)

Document End. This technical reference covers the complete architecture of the Commentary Engine as converted from Streamlit to FastAPI. For questions or updates, refer to the project README.md or contact the Genpact–Nissan Finance Technology team.